

TXSDK 主控交互指南



保密等级	A	TXSDK 主控交互指南	文件编号	TX-0000
发行日期	2024/05/07		文件版本	V1.0

责任与版权

责任限制

由于产品版本升级或者其他原因，本文档会不定期更新。除非另行约定，泰芯半导体有限公司对本文档所有内容不提供任何担保或授权。

客户应在遵守法律、法规和安全要求的前提下进行产品设计，并做充分验证。泰芯半导体有限公司对应用帮助或客户产品设计不承担任何义务。客户应对其使用泰芯半导体有限公司的产品和应用自行负责。

在适用法律允许的范围内，泰芯半导体有限公司在任何情况下，都不对因使用本文档相关内容及本文档描述的产品而产生的损失和损害进行超过购买支付价款的赔偿（除在涉及人身伤害的情况中根据适用的法律规定的损害赔偿外）。

版权申明

泰芯半导体有限公司保留随时修改本文档中任何信息的权利，无需提前通知且不承担任何责任。

未经泰芯半导体有限公司书面同意，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。除非获得相关权利人的许可，否则，任何人不能以任何形式对前述软件进行复制、分发、修改、摘录、反编译、反汇编、解密、反向工程、出租、转让、分许可等侵犯本文档描述的享有版权的软件版权的行为，但是适用法禁止此类限制的除外。



珠海泰芯半导体有限公司
Zhuhai TaiXin Semiconductor Co., Limited

珠海市高新区港湾一号科创园港
11 栋 3 楼

版权所有 侵权必究

Copyright © 2023 by TaiXin All Rights Reserved

保密等级	A	TXSDK 主控交互指南	文件编号	TX-0000
发行日期	2024/05/07		文件版本	V1.0

修订记录

日期	版本	描 述	修订人
2024-05-07	V1.0		TX

保密等级	A	TXSDK 主控交互指南	文件编号	TX-0000
发行日期	2024/05/07		文件版本	V1.0

目录

TXSDK 主控交互指南 1

1. 概述1

2. 系统初始化2

3. 数据接口3

4. 双协议栈4

5. WIFI_DHCPC 功能 5

6. PSALIVE 功能5

7. PSCONNECT 功能5

8. 自定义交互6

 8.1. 自定义命令 6

 8.1.1. 自定义 CMD ID 6

 8.1.2. HGIC_CMD_SET_CUST_DRIVER_DATA 8

 8.2. 自定义事件 8

1. 概述

TXSDK 是泰芯半导体发布的 WiFi/音视频系列芯片开发 SDK。本文档介绍的是 WiFi 模组与主控之间的交互行为，主要是针对低功耗 WiFi 模组。

WiFi 模组与主控之间交互的主要是网络数据，驱动命令，模组事件 3 类信息。这 3 类信息由 WiFi 驱动统一封装，主控端可以使用 Linux 驱动，RTOS 驱动或 NonOS 驱动。

本文档只介绍了 WiFi 模组上的开发方法，主控端的开发请查看对应的驱动开发指南文档。

2. 系统初始化

WiFi 模组 SDK 在初始化需要添加 wifi_mgr 模块的初始化，如下图所示。

wifi_mgr 模块负责与主控进行交互，包括数据收发，命令处理，事件上报。当主控未开机时，wifi_mgr 会缓存固件产生的事件和数据，待主控开机后再发送给主控。默认最多缓存 16 个事件和 16 个数据，超过最大值则丢弃旧的数据，缓存新的数据。

```
1:
2: __init static void sys_wifi_init(void)
3: {
4:     struct lmac_init_param lparam;
5:     struct ieee80211_initparam param;
6:
7:     skbpool_init(SKB_POOL_ADDR, (uint32)SKB_POOL_SIZE, 90, 0);
8:     os_memset(&lparam, 0, sizeof(lparam));
9:     lparam.rxbuf = WIFI_RX_BUFF_ADDR;
10:    lparam.rxbuf_size = WIFI_RX_BUFF_SIZE;
11:    lparam.tdma_buff = TDMA_BUFF_ADDR;
12:    lparam.tdma_buff_size = TDMA_BUFF_SIZE;
13:    Lmacops = lmac_ah_init(&lparam);
14:
15:    os_memset(&param, 0, sizeof(param));
16:    param.vif_maxcnt = 4;
17:    param.sta_maxcnt = sys_cfgs.sta_max ? sys_cfgs.sta_max : 8;
18:    param.bss_maxcnt = 32;
19:    param.bss_lifetime = 300; //300 seconds
20:    param.evt_cb = sys_ieee80211_event_cb;
21:
22:    ieee80211_init(&param);
23:    ieee80211_support_txw830x(Lmacops);
24:    wifi_mgr_init(FMAC_MAC_BUS, WIFIMGR_FRM_TYPE_HGIC, DRV_AGGSIZE, Lmacops, NULL);
25:
26:    ieee80211_deliver_init(128, 60);
27:
28: #if WIFI_AP_SUPPORT
29:     sys_wifi_ap_init();
30: #endif
31:
32: #if WIFI_STA_SUPPORT
33:     sys_wifi_sta_init();
34: #endif
35: }
```

wifi_mgr_init API 定义如下：

```
int32 wifi_mgr_init(enum mac_bus_type bus_type, int8 frm_type, uint16
drv_aggsize, void *ops, void *btops)
```

- 参数 bus_type [in]：和主控通信的接口类型，通常是 SDIO 或 USB
- 参数 frm_type [in]：和主控通信的帧类型
enum wifimgr_frm_type {
 WIFIMGR_FRM_TYPE_ETHER = 0, //以太网帧格式，只能交互以太网数据
 WIFIMGR_FRM_TYPE_HGIC, //泰芯驱动帧格式，可以交互数据，命令，事件
 WIFIMGR_FRM_TYPE_RAW, //裸数据格式，由固件完成以太网帧格式封装
};
- 参数 drv_aggsize [in]：接口聚合包 Size。0：关闭聚合，非 0：聚合包的 Size。

- 参数 ops [in]: lmac 协议栈句柄
- 参数 btops [in]: bt 协议栈句柄
- 返回值: 表示是否成功, 0: 成功, 非 0: 失败

根据方案低功耗需求, 可以选择初始化其他几个低功耗相关的模块:

- 初始化 psalive 功能: wifi_mgr_enable_psalive
- 初始化 psconnect 功能: wifi_mgr_enable_psconnect
- 初始化 dhcp client 模块: wifi_mgr_enable_dhcpc

3. 数据接口

WiFi 模组与主控可以使用多种接口进行交互, 例如 sdio, usb, uart, spi 等。为了方便支持多种接口, SDK 设计了 mac_bus 模块, 该模块定义了统一接口对接 wifi_mgr 模块。切换与主控通信接口时, 只需要修改宏定义 FMAC_MAC_BUS 的值。

SDK 所支持的接口代码在 sdk/lib/bus/mac_bus 目录下。

 combi_bus.c	2023/10/14 星期六 1...	C 文件
 gmac_bus.c	2023/10/24 星期二 1...	C 文件
 macbus.c	2024/5/11 星期六 16:...	C 文件
 mmc_bus.c	2024/5/11 星期六 16:...	C 文件
 ndev_bus.c	2023/10/14 星期六 1...	C 文件
 qc_bus.c	2023/10/14 星期六 1...	C 文件
 sdio_bus.c	2024/11/7 星期四 20:...	C 文件
 sdspi_bus.c	2024/5/11 星期六 16:...	C 文件
 spi_bus.c	2023/10/24 星期二 1...	C 文件
 spi_bus_8266.c	2023/10/14 星期六 1...	C 文件
 spi_bus_w600.c	2023/10/14 星期六 1...	C 文件
 uart_bus.c	2023/10/24 星期二 1...	C 文件
 usb_bus.c	2024/11/7 星期四 20:...	C 文件

4. 双协议栈

低功耗 WiFi 模组 SDK 支持启用 lwip 协议栈。这种情况下 WiFi 模组和主控同时运行各自的 tcpip 协议栈，主控中的网络接口（假设是 wlan0）和 WiFi 模组内的网络接口（假设是 w0），具有相同的 MAC 地址，相同的 IP 地址。WiFi 模组和主控各自运行独立的网络程序实现不同的功能。

wifi_mgr 模块会记录来自内部 lwip 协议栈的数据流，默认最多支持记录 16 路数据流，当从网络中接收到数据时会识别出该数据是送给内部 lwip 协议栈，还是送给主控协议栈。如果 SDK 提示“NO FREE LOCALID”时，表示记录的数据流已经超过最大数量。

主控的开机速度通常是比 WiFi 模组慢，可以由 WiFi 模块提前申请 IP 地址，主控开机后直接使用 WiFi 模组申请的 IP 地址，这样可以节省 DHCP 申请 IP 地址的时间。

在低功耗 WiFi 模组上运行的网络程序通常是休眠保活代码。设备进低功耗时主控通常会断电，WiFi 模组进入休眠状态。WiFi 模组周期性醒来执行保活代码，与服务器保持连接。

在 WiFi 模组上开发保活代码，和普通的网络程序开发没有区别。SDK 提供的 lwip 协议栈支持 BSD socket API，使用方法与其他系统一样。可以选择 tcp 保活或 udp 保活，按照设备的保活协议开发代码即可。建议在条件允许的情况下尽可能选择 udp 保活，udp 保活对功耗更加友好，保活代码逻辑控制也更加简单。

需要注意的是不同芯片的休眠行为有所不同，会影响休眠保活代码的开发方式。

- TXW8301 芯片由于不支持 XIP，代码在 SRAM 中运行。休眠时 SRAM 掉电，在唤醒时需要重新加载运行固件（相当于重新开机运行），这种情况不适合在 WiFi 模组上开发保活程序。
- TXW80x/TXW81x 芯片支持 XIP，代码在 flash 中运行。休眠唤醒时，系统可以恢复到休眠之前的状态继续运行，所有的状态信息都不会丢失。这种情况可以在 WiFi 模组上开发保活程序，模组周期性醒来执行保活程序。

5. WIFI_DHCP 功能

如果模组 SDK 不启用 lwip 协议栈，可以使用 wifi_dhcp 模块来申请 IP 地址。需要在系统初始化时使用 wifi_mgr_enable_dhcp API 初始化 wifi_dhcp 模块。

wifi_dhcp 模块可以在没有 tcpip 协议栈的情况下完成 dhcp 请求 IP 地址。申请 IP 地址后会自动通知主控。

对于 TXW8301 芯片来说，要启用这个模块，因为 TXW8301 芯片的程序代码是在 SRAM 中运行，不适合开发低功耗保活代码，这种情况下禁用 lwip，就可以节省 memory 资源。

6. PSALIVE 功能

PSALIVE 功能是 SDK 提供的泰芯自定义 UDP 保活机制。该功能简化了 UDP 保活功能开发，使用此功能进行低功耗保活时不需要开发保活代码，只需要设置一些参数即可。不过该功能使用上有限制条件：AP 和 STA 都必须是泰芯的 WiFi 芯片。

如果选择使用 PSALIVE 功能，在系统初始化时需要使用 wifi_mgr_enable_psalive API 初始 PSALIVE 模块。然后由主控设置相应参数，具体请阅读驱动开发指南的低功耗章节，选择 psmode 2。

7. PSCONNECT 功能

PSCONNECT 功能是为优化设备功耗而开发的一个模块。

该模块的工作原理：如果在休眠后出现断线情况，在不需要唤醒主控的情况下（由唤醒原因决定），WiFi 模组自行醒来连接 AP。如果未能连接 AP，模组则进入 psconnect 状态。psconnect 状态下 WiFi 模组开机后尝试连接一次 AP，连接失败则进入离线休眠状态，休眠时间 t 后自动唤醒再次尝试连接 AP，如果连接失败则休眠 2t 时间，依此规律逐渐加

大休眠时间，直到达到设定最大值后再回到初始时间 t ，依此循环。

psconnect 功能是为了避免在 AP 出现异常后，WiFi 模块频繁连接 AP 而增加功耗。该功能的参数设置请查看驱动开发指南文档。

8. 自定义交互

wifi_mgr 模块已经支持了驱动定义的命令和事件。但是应用开发时可能会需要添加一些自定义的命令或事件与主控进行交互。SDK 提供了添加自定义命令和事件的方法。

8.1. 自定义命令

添加自定义命令时需要和主控协同开发。目前有 2 种方式添加自定义命令，分别对应主控驱动的 2 个 API。

8.1.1. 自定义 CMD ID

推荐使用自定义 CMD ID 的方式添加自定义命令，请查看驱动的头文件 hgic.h（如下图所示）。

添加自定义命令时，可以直接在驱动的命令列表中新增 ID。建议为驱动预留足够的空间，自定义 ID 从 65000 开始，可以添加 535 个命令，应该是足够应用程序使用了。

```

HGIC_CMD_SET_WIFI_RPASSWD = HGIC_CMD_START + 179, /* fmac */
HGIC_CMD_GET_WIFI_UUID    = HGIC_CMD_START + 180, /* fmac/smac */
HGIC_CMD_SET_COEXIST_EN   = HGIC_CMD_START + 181, /* smac */
HGIC_CMD_SET_THROUGHPUT_EN = HGIC_CMD_START + 182, /* smac */
HGIC_CMD_SET_PS_HBDATA_MASK = HGIC_CMD_START + 183, /* fmac */
HGIC_CMD_SET_MEM_RECYCLE  = HGIC_CMD_START + 184, /* smac/fmac */
HGIC_CMD_SET_HOST_MONITOR = HGIC_CMD_START + 185, /* smac/fmac */
HGIC_CMD_SET_DMA_SCATTER  = HGIC_CMD_START + 186, /* smac/fmac */
HGIC_CMD_SET_BLE_LLPKT_LEN = HGIC_CMD_START + 187, /* smac/fmac */
HGIC_CMD_SET_SLEEP_ROAMING = HGIC_CMD_START + 188, /* fmac */
HGIC_CMD_SET_EDCA_LIMIT_MAX = HGIC_CMD_START + 189, /* smac/fmac */
HGIC_CMD_SET_EDCA_SLOT_TIME = HGIC_CMD_START + 190, /* smac/fmac */

/*add customer driver cmd, recommend start from 65000, max (65535 - HGIC_CMD_START).*/
//////////////////////////////////////////////////
/*virtual driver cmd, exceed 65535*/
HGIC_CMD_SEND_BLENC_HCI_DATA = HGIC_CMD_START + 0xFFFF + 1, /*BLE HCI DATA FOR RTOS DRI
} « end hgic_cmd » ;
```

使用自定义 CMD ID 时，在主控端使用 `hgic_iwpriv_set_customer_dvrdata` API 发送命令。

```
int hgic_iwpriv_set_customer_dvrdata(char *ifname, unsigned short cmd_id, char *data, int len)
```

- 参数 ifname: 接口名称
- 参数 cmd_id: 自定义的 cmd_id
- 参数 data: 命令数据
- 参数 len: 命令数据长度
- 返回值: 表示是否成功。0: 成功, 非 0: 失败

使用自定义 CMD ID 时，在 WiFi 模组端需要使用 `wifi_proc_drvcmd_cust` 函数处理这些自定义命令。在 main.c 中已经提供了 `wifi_proc_drvcmd_cust` 函数，在该函数添加对应的处理代码即可。

wifi_mgr 模块接收的所有来自主控的命令都会先执行 `wifi_proc_drvcmd_cust` 函数，如果某个命令被 `wifi_proc_drvcmd_cust` 函数处理过了（返回值为 `RET_OK`），wifi_mgr 则忽略该 cmd 的后续处理。返回 `-ENOTSUPP` 表示 `wifi_proc_drvcmd_cust` 函数未处理该 cmd，wifi_mgr 模块继续处理。

```
int32 wifi_proc_drvcmd_cust(uint16 cmd_id, uint8 *data, uint32 len, void *hdr)
{
    /*
     * cmd_id: driver cmd id (from host driver).
     * data  : cmd data.
     * len   : cmd data length.
     * hdr   : cmd info, just used to call host_cmd_resp API.
     *
     * 1. parse cmd data, do some thing if you need.
     *
     * 2. send cmd response to host:
     *    // has no additional response data.
     *    host_cmd_resp(return_value, NULL, 0, hdr);
     *
     *    // has some additional response data.
     *    host_cmd_resp(return_value, reponse_data, response_data_length, hdr);
     *
     * 3. return RET_OK if the driver cmd has been processed, else return -ENOTSUPP.
     *
     * you can process any driver cmd in this function.
     */
    return -ENOTSUPP;
} « end wifi_proc_drvcmd_cust »
```

8.1.2. HGIC_CMD_SET_CUST_DRIVER_DATA

早期的驱动定义了一个特殊的 CMD ID: `HGIC_CMD_SET_CUST_DRIVER_DATA`, 用于传输自定义驱动数据。如果主控程序使用了 `hgic_iwpriv_set_cust_driverdata` API, 使用的就是这个 ID。这种情况下, 需要在 `wifi_proc_drvcmd_cust` 函数中添加对 `HGIC_CMD_SET_CUST_DRIVER_DATA` 的处理, 解析处理自定义的数据。

8.2. 自定义事件

应用程序可以添加自定义事件通知主控。类似的在驱动 event ID 列表中添加自定义 ID 即可, 如下图所示:

```
471: HGIC_EVENT_UNLINK_STA = 24,
472: HGIC_EVENT_BLENC_DATA = 25,
473: HGIC_EVENT_HMSCAN_RESULT = 26,
474: HGIC_EVENT_EXCEPTION_INFO = 27,
475: HGIC_EVENT_DSLEEP_WAKEUP = 28,
476: HGIC_EVENT_STA_MIC_ERROR = 29,
477: HGIC_EVENT_ACS_DONE = 30,
478: HGIC_EVENT_FW_INIT_DONE = 31,
479: HGIC_EVENT_ROAM_CONNECTED = 32,
480: HGIC_EVENT_MGMT_FRAME = 33,
481: HGIC_EVENT_UNKNOWN_STA = 34,
482: HGIC_EVENT_ROAM_FAIL = 35,
483:
484: /*add customer driver event, recommend start from 65000, max 65535.*/
485:
486: ///////////////////////////////////////////////////
487: /*virtual driver event, exceed 65535*/
488: HGIC_EVENT_HGIC_DATA = 0xFFFF + 1, /*REPORT DRIVER HGIC DATA FOR RTOS DRIVER*/
489: } « end hgic_event » ;
490:
```

建议自定义事件 ID 从 65000 开始, 为驱动预留足够的空间。

在 WiFi 模组中使用 `host_event_new` API 产生事件并通知主控。在主控端的 demo 代码 `hgicf.c` 文件中的 `hgicf_fwevent_parse` 函数添加自定义 ID 处理代码即可。如下图所示:

```

5:
6: uint hgicf_fwevent_parse(u8 *event_data, u32 event_len)
7: {
8:     int i = 0;
9:     u32 data_len = 0;
10:    u32 evt_id = 0;
11:    char *data = NULL;
12:    char *buff;
13:    struct hgic_exception_info *exp;
14:    struct hgic_dhcp_result *dhcpc;
15:    struct hgic_ctrl_hdr *evt = (struct hgic_ctrl_hdr *)event_data;
16:
17:    data = (char *)(evt + 1);
18:    data_len = event_len - sizeof(struct hgic_ctrl_hdr);
19:    evt_id = HDR_EVTID(evt);
20:
21:    //printf("recv firmware event %d\r\n", evt_id);
22:    buff = malloc(4096);
23:    if (buff == NULL) { ...
24:
25:
26:    switch (evt_id) {
27:        case HGIC_EVENT_SCANNING: ...
28:        case HGIC_EVENT_SCAN_DONE: ...
29:        case HGIC_EVENT_TX_BITRATE: ...
30:        case HGIC_EVENT_PAIR_START: ...
31:        case HGIC_EVENT_PAIR_SUCCESS: ...
32:        case HGIC_EVENT_PAIR_DONE: ...
33:        case HGIC_EVENT_CONNECT_START: ...
34:        case HGIC_EVENT_CONNECTED: ...
35:        case HGIC_EVENT_ROAM_CONNECTED: ...
36:        case HGIC_EVENT_DISCONNECTED: ...
37:        case HGIC_EVENT_SIGNAL: ...
38:        case HGIC_EVENT_CUSTOMER_MGMT: ...
39:        case HGIC_EVENT_DHCPC_DONE: ...
40:        case HGIC_EVENT_CONNECT_FAIL: ...
41:        case HGIC_EVENT_CUST_DRIVER_DATA: ...
42:        case HGIC_EVENT_UNPAIR_STA: ...
43:        case HGIC_EVENT_FWDBG_INFO: // 固件调试信息输出 ...
44:        case HGIC_EVENT_EXCEPTION_INFO: ...
45:        case HGIC_EVENT_BLENC_DATA: //process BLE Advertising data. BLE work mode 1 or 2. ...
46:
47:        free(buff);
48:    } // « end hgicf_fwevent_parse »
49:
50:

```

host event new API 说明如下:

```
uint32 host event new(uint32 evt_id, uint8 *data, uint32 data_len)
```

- 参数 evt_id : 事件 ID
- 参数 data : 事件数据
- 参数 data_len: 数据长度